Algorithm 1 — Hybrid A (PFD-GSTE Variant A) Forward Pass

Input: image batch x, flag return_xai
Output: logits, and optionally {"mask": mask_img, "attn": attn_list}

1. feat ← CNN(x) (ResNet50V2 feature map, e.g., (B,2048,7,7))
2. z_cnn ← AvgPool(feat)
3. z_cnn ← Flatten(z_cnn)
4. z_cnn ← ReLU(Linear_cnn_proj(z_cnn))
5. (feat_path, mask_feat) ← PFD(feat)
    - mask_feat ← Sigmoid(Conv1x1(feat))
    - feat_path ← feat * mask_feat
6. mask_img ← BilinearUpsample(mask_feat, size = input_image_size) (only for visualisation/XAI)
7. token_sets ← empty list
8. **For each** (rot_idx, k) **in** enumerate(rotations) **do**
   8.1 f_r ← rot90(feat_path, k)
   8.2 m_r ← rot90(mask_feat, k)
   8.3 (tokens, ht, wt) ← FeatureTokenEmbed(f_r)
   8.4 alpha ← Flatten(m_r) as (B, ht*wt, 1)
   8.5 alpha ← alpha / (mean(alpha over tokens) + 1e-6)
   8.6 tokens ← tokens * alpha (GSTE-A: static guidance weighting)
   8.7 tokens ← PosRotEmbed(tokens, ht, wt, rot_idx)
   8.8 append tokens to token_sets
   Endfor
9. Tavg ← Mean(token_sets over rotations)
10. (Tenc, attn_list) ← RViTEncoder(Tavg, return_attn = return_xai)
11. z_vit ← Mean(Tenc over tokens)
12. z_vit ← ReLU(Linear_vit_proj(z_vit))
13. z ← Concat(z_cnn, z_vit)
14. h ← ReLU(Linear_fuse_fc(z))
15. h ← Dropout(h)
16. logits ← Linear_out(h)
17. **If** return_xai **then**
    Return logits, {"mask": mask_img, "attn": attn_list}
    Else
    Return logits, None
    Endif

Algorithm 2 — Hybrid B (PFD-GSTE Variant B) Forward Pass

Input: image batch x, flag return_xai
Output: logits, and optionally {"attn": attn_list, "mask": mask_img, "gste_side": dyn_side}

1. feat ← CNN(x) (ResNet50V2 feature map, e.g., (B,C,7,7))
2. **If** use_pfd_gste == True **then**
   2.1 (feat_path, mask_feat) ← PFD(feat)
   2.2 feat_for_cnn ← feat_path
   2.3 mask_img ← BilinearUpsample(mask_feat, size = input_image_size)
   Else
   2.4 feat_for_cnn ← feat
   2.5 mask_img ← None
   Endif
3. cache _last_cnn_feat ← feat_for_cnn
4. cache _last_cnn_mask_img ← mask_img
5. **CNN descriptor head**
   5.1 z_cnn ← AvgPool(feat_for_cnn)
   5.2 z_cnn ← Flatten(z_cnn)
   5.3 z_cnn ← Dropout(z_cnn)
   5.4 z_cnn ← Linear_cnn_proj(z_cnn)
   5.5 z_cnn ← BatchNorm(z_cnn)
   5.6 z_cnn ← ReLU(z_cnn)
6. **Choose dynamic token grid side (GSTE decision)**
   If use_pfd_gste == True **then**
   6.1 alpha0 ← AdaptiveAvgPool(mask_img, output=(g,g)) (g = base_grid = img_size//patch_size)
   6.2 alpha0 ← alpha0 / (mean(alpha0 over grid) + 1e-6)
   6.3 dyn_side ← choose_dynamic_side(alpha0) where:
   - s ← mean(std(alpha0 over grid cells))
   - **If** s ≤ gste_std_flat **then** dyn_side = g
   - **Else**
     - t ← clamp((s - gste_std_flat)/(gste_std_full - gste_std_flat), 0, 1)
     - shrink ← gste_max_shrink * t
     - target ← round(g * (1 - shrink))
     - dyn_side ← clamp(target, gste_min_side, g)
       Else
       6.4 dyn_side ← g
       Endif
7. token_sets ← empty list
8. **For each** k **in** rotations **do**
   8.1 rot_id ← k mod 4
   8.2 x_r ← rot90(x, rot_id)
   8.3 (tokens_base, ht, wt) ← ImagePatchEmbed(x_r) (tokens on g×g grid)

   8.4 **If** use_pfd_gste == True **then**
   8.4.1 m_r ← rot90(mask_img, rot_id)
   8.4.2 alpha_base ← AdaptiveAvgPool(m_r, output=(g,g))
   8.4.3 alpha_base ← alpha_base / (mean(alpha_base over grid) + 1e-6)

   8.4.4 (tokens, ht, wt) ← GSTE_dynamic_tokens(tokens_base, alpha_base, dyn_side) where:

   - reshape tokens_base → (B,D,g,g)
   - multiply by alpha_base (ROI weighting)
   - **If** dyn_side < g **then** weighted pool to (dyn_side,dyn_side) using (num/den)
   - flatten back → (B, dyn_side*dyn_side, D)
     Else
     8.5 tokens ← tokens_base
     Endif

8.6 tokens ← PosRotEmbed(tokens, ht, wt, rot_id)
8.7 append tokens to token_sets
Endfor

9. Tavg ← Mean(token_sets over rotations)
10. (Tenc, attn_list) ← RViTEncoder(Tavg, return_attn = return_xai)
11. z_vit ← Mean(Tenc over tokens)
12. z_vit ← Linear_vit_proj(z_vit)
13. z_vit ← ReLU(z_vit)
14. z ← Concat(z_cnn, z_vit)
15. h ← Linear_fuse_fc(z)
16. h ← ReLU(h)
17. h ← Dropout(h)
18. logits ← Linear_out(h)
19. **If** return_xai **then**
Return logits, {"attn": attn_list, "mask": mask_img, "gste_side": dyn_side}
Else
Return logits, None
Endif